

MemPoisonGuard: A Provenance-Aware Framework for Detecting and Mitigating Long-Term Memory Poisoning in AI Agents

Premkumar S

Student

M.Tech Data Science

Rajalakshmi Engineering College

Chennai, Tamil Nadu, India

prem021003@gmail.com

Dr. Prasannakumari V

Professor

Department of Information Technology

Rajalakshmi Engineering College

Chennai, Tamil Nadu, India

prasannakumari.v@rajalakshmi.edu.in

Abstract—Long-term memory enables AI agents to preserve user preferences, facts, and task context across sessions; however, it also creates a persistent attack surface. Malicious instructions, false claims, or unverified authorization statements can be stored as memory and later influence responses to otherwise benign queries. This paper presents MemPoisonGuard, a locally runnable provenance-aware framework for detecting and mitigating long-term memory poisoning in AI agents. The framework evaluates candidate memories before persistence through source provenance capture, SHA-256 content hashing, rule-based prompt-injection detection, sensitive authorization-claim screening, natural language inference-based contradiction analysis, and trust-risk scoring. Each memory candidate is classified as verified, provisional, or quarantined. During response generation, only verified memory is eligible for semantic retrieval and is reranked using relevance, trust, and risk signals. A controlled evaluation of five benign and seven malicious memory candidates compares an unprotected baseline with MemPoisonGuard. In the reported controlled experiment, the baseline stored all malicious candidates, whereas MemPoisonGuard quarantined six and marked one as provisional, preventing all malicious candidates from entering the verified-memory retrieval collection. These results are limited to the controlled evaluation set and do not establish universal robustness.

Keywords—AI agent security, long-term memory, memory poisoning, prompt injection, provenance, secure retrieval

I. INTRODUCTION

AI agents increasingly combine large language models with persistent memory to retain user preferences, task context, historical observations, and retrieved knowledge across multiple interactions [1], [2]. Although this capability improves continuity and personalization, it also creates a persistent attack surface. Malicious instructions, misleading facts, and unverified authorization claims can be stored during one interaction and later retrieved during an unrelated conversation.

Memory poisoning differs from one-session prompt injection because adversarial content can persist beyond the original interaction [1], [4]. A poisoned memory may later influence retrieval, agent reasoning, or response generation. Therefore, memory-enabled agents require controls not only at the prompt boundary but also at the memory-write and memory-retrieval stages.

This paper presents MemPoisonGuard, a provenance-aware framework for validating candidate memories before they become persistent agent state. The framework captures source provenance and a SHA-256 content hash, applies rule-based prompt-injection detection, identifies sensitive authorization claims, detects contradictions against trusted verified memory using natural language inference, and computes trust and risk scores. Based on these signals, each memory is classified as verified, provisional, or quarantined. Only verified memory is eligible for guarded retrieval.

The main contributions of this work are as follows:

1. A provenance-aware memory-write validation framework for local AI agents.
2. A layered defense combining injection detection, sensitive-claim screening, NLI contradiction analysis, and trust-risk scoring.
3. A three-state memory governance model consisting of verified, provisional, and quarantined memory.
4. A verified-only retrieval mechanism with relevance, trust, and risk-aware ranking.
5. A controlled baseline-versus-guarded evaluation using benign and malicious memory candidates.

The remainder of this paper is organized as follows. Section II reviews related work. Section III presents the proposed framework. Section IV describes the experimental setup. Section V discusses the controlled evaluation results. Section VI concludes the paper and identifies future work.

II. RELATED WORK

Persistent memory is increasingly used in AI agents to retain facts, user preferences, task context, and historical interactions across sessions. However, persistent storage also expands the attack surface because untrusted content can be retained and later presented to the language model as contextual information [1], [2]. In contrast to a conventional prompt-injection attack that is typically limited to an active interaction [1], [4], memory poisoning separates the malicious write event from its eventual effect. A single successful write can therefore influence later interactions without requiring the original payload to remain in the active prompt.

Recent studies have examined memory poisoning in long-term-memory-enabled agents. These studies show that adversarial content can enter memory through normal interaction channels and subsequently affect agent behavior during later sessions [1], [2]. The threat is particularly difficult because some malicious memories may resemble plausible user preferences, policies, facts, or prior observations rather than explicit instructions. Consequently, filtering only obvious injection phrases at the initial prompt boundary is insufficient for protecting persistent memory.

Prompt injection and retrieval security are closely related to memory poisoning. Retrieval-augmented systems commonly select context using semantic similarity, which can cause attacker-controlled content to be retrieved when it is crafted to appear relevant to a future query. If a system treats retrieved content as trusted context without considering its origin, integrity, or consistency with existing knowledge, poisoning can affect downstream reasoning and responses [5]. These observations motivate security controls at both the memory-write stage and the retrieval stage.

Provenance-aware approaches provide a useful basis for memory governance. Recording a memory item’s source, creation time, session context, integrity hash, and validation state enables later auditing and supports differentiated handling of trusted and untrusted content. Rather than treating every stored item as equally reliable, a memory system can assign a trust state and restrict high-risk entries from being used as authoritative context [7], [8].

Natural language inference (NLI) offers an additional signal for identifying contradictory candidate memories. NLI models estimate whether a hypothesis is entailed by, contradicted by, or neutral with respect to a premise [9]. In a memory-security setting, a candidate memory can be evaluated against relevant verified memory items to identify semantic conflicts that are not captured by exact string matching.

Existing work establishes that persistent memory poisoning is a meaningful agent-security problem, but practical local defenses must combine several signals. MemPoisonGuard addresses this gap through a lightweight, provenance-aware pipeline that combines rule-based injection screening, sensitive-authorization-claim detection, NLI-based contradiction analysis, trust-risk scoring, quarantine, and verified-only retrieval [2], [7].

III. PROPOSED MEMPOISONGUARD FRAMEWORK

MemPoisonGuard is a local, provenance-aware memory-security framework for AI agents that use long-term memory. Its objective is not to determine whether every candidate memory statement is universally true. Instead, it assesses whether a candidate should be trusted for persistent storage and later retrieval. The framework evaluates each candidate during the memory-write stage and assigns one of three states: verified, provisional, or quarantined. Only verified memories are eligible for ordinary semantic retrieval.

Fig. 1 presents the overall workflow. A candidate memory is received from an interaction source and is first normalized for analysis. The framework records provenance metadata and computes an integrity hash. It then applies layered checks for prompt-injection patterns, sensitive authorization claims, and semantic contradiction with relevant verified memories. The

resulting signals are combined into risk and trust scores, which drive the final storage decision.

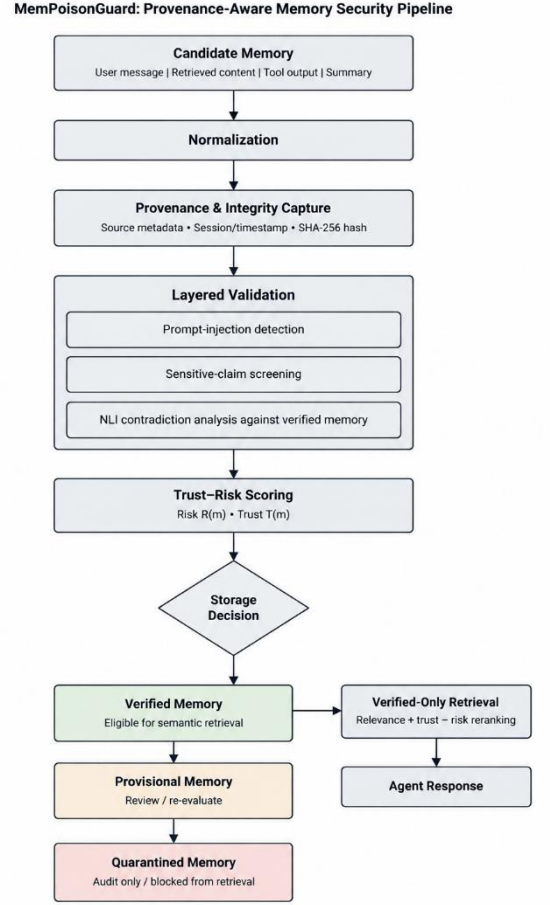


Fig. 1. MemPoisonGuard workflow: candidate memory ingestion, normalization, provenance and hashing, injection and sensitive-claim screening, NLI contradiction analysis, trust-risk scoring, state assignment, and verified-only retrieval.

A. Threat Model and Design Goals

The considered adversary attempts to influence a future agent response by causing malicious, deceptive, or unauthorized content to be retained as long-term memory. Candidate memories may originate from user messages, retrieved content, tool outputs, or application-generated summaries. The threat model includes direct prompt-injection instructions, attempts to bypass security controls, false authorization statements, data-exfiltration requests, contradictory facts, and retrieval-manipulation content.

MemPoisonGuard is designed to reduce the probability that such content becomes trusted persistent context. Its design goals are to preserve useful benign memories, retain sufficient metadata for auditing, isolate suspicious content from verified retrieval, and provide an interpretable reason for each storage decision. The framework does not claim to guarantee detection of all adaptive, multilingual, obfuscated, or previously unseen attacks [6], [7].

B. Provenance and Integrity Capture

For every candidate memory m , MemPoisonGuard records a structured provenance record containing a unique memory identifier, source type, source session or interaction identifier, creation time, normalized content, validation signals, classification state, and a SHA-256 content hash. The hash provides an integrity reference that can be recomputed later to identify unintended modifications to stored content [10].

The provenance record enables later audit of why a memory was accepted, provisionally retained, or quarantined. It also avoids treating all persisted content as equally trustworthy. A memory received directly from an authenticated and expected source can receive a higher provenance signal than an unauthenticated or externally retrieved item; however, provenance alone does not override evidence of injection or contradiction.

$$h(m) = \text{SHA} - 256(\text{normalize}(m)) \quad (1)$$

Here, $\text{normalize}(\cdot)$ denotes deterministic preprocessing applied before hashing, and $h(m)$ is the stored integrity digest. SHA-256 is a standardized hash algorithm that produces a message digest suitable for detecting changes to a message after the digest is generated.

C. Injection and Sensitive-Claim Screening

The first security layer applies deterministic rules to the normalized candidate text. The injection detector searches for patterns associated with instruction override, role reassignment, system-prompt extraction, security bypass, tool misuse, or hidden instruction execution. The sensitive-claim detector separately identifies statements that assert privileged authorization, policy exceptions, credential disclosure, unrestricted access, or other high-impact permissions.

Rule-based screening is intentionally transparent and inexpensive for a local deployment [6], [7]. It is not used as the sole decision mechanism because attackers may paraphrase or obfuscate malicious content. Instead, positive matches increase the candidate risk score and contribute to a quarantine decision when combined with other evidence.

D. NLI-Based Contradiction Analysis

To identify semantic inconsistency, MemPoisonGuard compares a candidate memory with relevant verified memory items. Candidate-memory pairs are evaluated using a natural language inference model that estimates whether the candidate is entailed by, contradicted by, or neutral with respect to an existing verified memory [9]. The contradiction probability is retained as a signal rather than interpreted as conclusive proof of falsehood.

Let $p_{\text{contr}}(m, v)$ denote the contradiction probability...between candidate memory m and a relevant verified memory v . The framework uses the maximum contradiction probability across the retrieved comparison set $V(m)$:

$$c(m) = \max_{v \in V(m)} p_{\text{contr}}(m, v) \quad (2)$$

A high value of $C(m)$ indicates that the candidate conflicts semantically with at least one relevant verified memory. The system can then place the candidate in quarantine or require further review rather than allowing it to overwrite trusted

context. NLI is appropriate for this role because it explicitly models entailment, contradiction, and neutral relationships between premise and hypothesis.

E. Trust-Risk Scoring and Decision Policy

The framework combines multiple validation signals into interpretable risk and trust scores. Let $I(m)$ be the injection indicator, $A(m)$ be the sensitive-authorization-claim indicator, $P(m)$ be the provenance confidence, and $C(m)$ be the NLI contradiction score. Each signal is normalized to the interval $[0,1]$. The risk score is defined as:

$$R(m) = 0.50I(m) + 0.20C(m) + 0.20[1 - P(m)] + 0.10A(m) \quad (3)$$

The trust score rewards reliable provenance, successful validation, and the absence of injection and contradiction evidence:

$$T(m) = 0.45P(m) + 0.25V(m) + 0.20[1 - C(m)] + 0.10[1 - I(m)] \quad (4)$$

In (4), $V(m)$ denotes the validation confidence produced by the local validation pipeline. The weights are design choices for the controlled prototype and can be tuned using a larger validation dataset. The storage decision policy is:

$$\text{state}(m) = \begin{cases} \text{quarantined}, & R(m) \geq \tau_R \vee I(m) = 1 \\ \text{verified}, & T(m) \geq \tau_T \wedge R(m) < \tau_R \\ \text{provisional}, & \text{otherwise} \end{cases} \quad (5)$$

Here, τ_R and τ_T are configurable risk and trust thresholds. Quarantined entries are retained separately for audit but are excluded from verified retrieval. Provisional entries can be reviewed later or re-evaluated when additional evidence becomes available.

F. Verified-Only Retrieval

At query time, MemPoisonGuard retrieves semantic candidates only from the verified-memory collection. This isolation prevents quarantined and provisional entries from being automatically supplied as trusted context to the language model. Retrieved verified candidates are reranked using semantic relevance, trust, and risk:

$$S_{\text{safe}}(m, q) = 0.65 \text{Rel}(m, q) + 0.25T(m) - 0.10R(m) \quad (6)$$

In (6), $\text{Rel}(m, q)$ is the semantic relevance of memory m to query q . The ranking function prioritizes memories that are both relevant and well validated while penalizing residual risk. This design separates storage eligibility from retrieval ranking: a memory must first become verified, and only then can it compete for retrieval based on relevance and safety signals.

IV. EXPERIMENTAL SETUP

A. Evaluation Objective and Scenarios

The controlled evaluation assesses whether MemPoisonGuard prevents malicious memory candidates

from entering the verified-memory collection while retaining benign memories for normal retrieval. The evaluation compares an unprotected baseline agent, which stores submitted candidates directly, with the guarded MemPoisonGuard workflow, which validates candidates before storage.

The evaluation set contains 12 manually defined memory candidates: five benign candidates and seven malicious candidates. Benign candidates represent ordinary preferences, factual notes, and task-related context that should remain available to the agent. Malicious candidates represent direct prompt injection, security-bypass attempts, data-exfiltration requests, authority spoofing, contradiction poisoning, false authorization claims, and retrieval-manipulation attempts. The set was designed as a controlled functional-security test rather than as a representative benchmark of real-world attack prevalence.

CATEGORY	COUNT	EXPECTED OUTCOME
BENIGN MEMORY	5	VERIFIED
PROMPT INJECTION	1	QUARANTINED
SECURITY BYPASS	1	QUARANTINED
DATA EXFILTRATION	1	QUARANTINED
AUTHORITY SPOOFING	1	PROVISIONAL OR QUARANTINED
CONTRADICTION MEMORY	1	QUARANTINED
FALSE AUTHORIZATION	1	QUARANTINED
RETRIEVAL MANIPULATION	1	QUARANTINED

TABLE I. CONTROLLED MEMORY-CANDIDATE SCENARIOS

B. Systems Compared

Two systems were evaluated. The baseline system accepts each candidate memory and stores it without provenance-aware validation, injection screening, contradiction analysis, trust-risk scoring, or state-based retrieval restrictions. The MemPoisonGuard system processes every candidate through normalization, provenance and SHA-256 integrity capture, rule-based injection detection, sensitive-claim screening, NLI-based contradiction analysis, risk-trust scoring, and state assignment.

For the guarded system, candidates classified as verified are stored in the verified-memory collection and are eligible for semantic retrieval. Candidates classified as provisional are retained separately for later review or re-evaluation. Candidates classified as quarantined are retained only for audit and are excluded from verified-memory retrieval.

C. Implementation Environment

The prototype was implemented and evaluated on a local Windows system with an Intel Core i5-10310U CPU operating at 1.70 GHz and 16 GB of RAM. No dedicated GPU was used. The software environment used Python 3.12.10, FastAPI for the backend service [12], SQLite for structured local records, ChromaDB for vector-based memory storage and retrieval [13], Sentence Transformers for embeddings and NLI-related model integration, Streamlit for the dashboard, Pandas and scikit-learn for analysis, and Plotly for visualization.

A local Ollama deployment running the qwen2.5:3b model [11] was used for language-model interaction. All experiments were conducted locally. The reported results therefore reflect the described prototype, implementation settings, validation rules, and controlled candidate set; they should not be interpreted as hardware-independent or model-independent performance estimates.

D. Evaluation Metrics

The primary evaluation objective is to identify malicious memory candidates before they enter verified retrieval. A malicious candidate correctly blocked from verified memory is treated as a true positive, whereas a malicious candidate incorrectly accepted as verified is treated as a false negative. A benign candidate correctly accepted as verified is treated as a true negative, whereas a benign candidate incorrectly blocked or quarantined is treated as a false positive.

Precision, recall, and F1-score are calculated for malicious-candidate detection. The false-positive rate and false-negative rate are also reported to indicate the effect of the safeguards on benign content and the residual risk of allowing malicious content into verified memory. Storage outcomes, including verified, provisional, and quarantined counts, are reported alongside the classification metrics.

$$Precision = \frac{TP}{TP + FP} \quad (7)$$

$$Recall = \frac{TP}{TP + FN} \quad (8)$$

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (9)$$

Here, TP, FP, TN, and FN denote true positives, false positives, true negatives, and false negatives, respectively. For the controlled result table, a provisional malicious candidate is counted as successfully withheld from verified memory because it is not eligible for ordinary verified retrieval.

E. Ethical and Security Considerations

The evaluation uses only synthetic, non-sensitive memory candidates created for controlled security testing. No personal data, credentials, private documents, or active external systems were targeted. Malicious examples are included solely to assess detection and containment behavior and are not executed as instructions.

MemPoisonGuard is intended as a defense-in-depth prototype rather than a substitute for access control, source authentication, secure tool design, model alignment, audit

logging, or human review. Rule-based detection and NLI analysis can produce errors, particularly for ambiguous, multilingual, adversarially paraphrased, or domain-specific content. The system therefore retains provisional and quarantined entries for audit instead of silently deleting them.

V. RESULTS AND DISCUSSION

A. Storage Outcomes

The controlled evaluation processed 12 memory candidates: five benign candidates and seven malicious candidates. The unprotected baseline stored all submitted candidates directly and therefore accepted all seven malicious candidates into its persistent memory. Because the baseline did not maintain trust states or retrieval restrictions, these malicious entries remained available for later retrieval.

MemPoisonGuard evaluated each candidate before persistence. All five benign candidates were classified as verified. Of the seven malicious candidates, six were classified as quarantined and one authority-spoofing candidate was classified as provisional. Consequently, no malicious candidate entered the verified-memory collection. Since provisional and quarantined entries are excluded from ordinary verified retrieval, the guarded workflow withheld all seven malicious candidates from use as trusted context.

TABLE II. BASELINE AND MEMPOISONGUARD STORAGE OUTCOMES

Outcome	Baseline	MemPoisonGuard
Total candidates processed	12	12
Benign candidates available for retrieval	5	5
Malicious candidates stored as verified	7	0
Malicious candidates quarantined	0	6
Malicious candidates provisional	0	1

Table II shows that the baseline and guarded systems differ primarily at the memory-write boundary. The baseline has no mechanism to distinguish trustworthy context from adversarial or unverified content. In contrast, MemPoisonGuard assigns an explicit state to each candidate and prevents non-verified entries from entering the retrieval path used for ordinary agent responses.

B. Detection Metrics

For malicious-candidate detection, a candidate withheld from verified memory was treated as a successful detection. The guarded prototype produced seven true positives, zero false positives, five true negatives, and zero false negatives on the controlled set. This produces a precision of 1.0000, a recall of 1.0000, an F1-score of 1.0000, a false-positive rate of 0.0000, and a false-negative rate of 0.0000.

TABLE III. CONTROLLED EVALUATION METRICS

Metric	Baseline	MEMPOISONGUARD
TOTAL SCENARIOS	12	12
BENIGN SCENARIOS	5	5
MALICIOUS SCENARIOS	7	7
PRECISION	N/A	1.0000
RECALL	N/A	1.0000
F1-SCORE	N/A	1.0000
FALSE-POSITIVE RATE	N/A	0.0000
FALSE-NEGATIVE RATE	N/A	0.0000

The baseline values are shown as not applicable because the baseline does not perform malicious-memory classification or apply an explicit security decision policy. The guarded-system values summarize performance only on the defined controlled candidate set.

C. Retrieval Isolation

The storage-state design provided a second layer of protection after detection. Quarantined entries were retained for audit but excluded from verified retrieval. The provisional authority-spoofing entry was also excluded from ordinary verified retrieval, preventing an uncertain authorization claim from being supplied to the language model as trusted context.

This separation is important because identifying suspicious content is not sufficient if the system still retrieves that content during later agent operation [3], [6]. By restricting semantic retrieval to the verified-memory collection and then reranking candidates with relevance, trust, and risk signals, MemPoisonGuard reduces the likelihood that malicious or insufficiently validated context affects response.

D. Interpretation and Limitations

The results demonstrate that, under the specified local implementation and controlled 12-scenario evaluation, MemPoisonGuard prevented all seven malicious candidates from entering verified retrieval while retaining all five benign candidates as verified memory. The result supports the feasibility of combining provenance capture, deterministic screening, contradiction analysis, trust-risk scoring, quarantine, and verified-only retrieval in a local AI-agent memory pipeline.

The reported precision, recall, and F1-score values of 1.0000 were obtained only on the defined controlled 12-scenario dataset. They do not establish universal robustness against real-world, multilingual, obfuscated, adaptive, or

previously unseen memory-poisoning attacks. The dataset is small, manually constructed, and may not capture realistic attacker strategies, ambiguous inputs, model errors, source-authentication failures, or retrieval behavior under large memory collections.

The prototype also depends on threshold values, rule definitions, embedding behavior, and NLI-model outputs. These components can produce false positives or false negatives in broader deployment settings. Future evaluation should use larger and independently constructed datasets, multiple attack paraphrases, multilingual examples, external retrieved content, varied models, ablation studies, and human-in-the-loop review of provisional decisions [8].

VI. CONCLUSION AND FUTURE WORK

Persistent memory improves the continuity and usefulness of AI agents, but it also creates a security risk when untrusted content can be retained and later retrieved as context. This paper presented MemPoisonGuard, a provenance-aware framework for reducing long-term memory poisoning in local AI agents. The framework validates candidate memories before persistence using provenance capture, SHA-256 integrity hashing, rule-based prompt-injection and sensitive-claim screening, NLI-based contradiction analysis, trust-risk scoring, and a three-state storage policy consisting of verified, provisional, and quarantined memory.

In the controlled evaluation of 12 manually defined memory candidates, the unprotected baseline stored all seven malicious candidates directly. MemPoisonGuard classified five benign candidates as verified, quarantined six malicious candidates, and marked one malicious authority-spoofing candidate as provisional. Thus, no malicious candidate entered verified retrieval in the reported experiment. These results demonstrate the feasibility of applying layered, interpretable checks at the memory-write boundary and of separating suspicious content from the retrieval path used for ordinary agent responses.

The results are limited to the local prototype and the defined controlled dataset. Future work should evaluate the framework using larger and independently constructed datasets, indirect prompt-injection sources, external documents and web retrieval, multilingual and obfuscated attacks, multiple language and NLI models, threshold

calibration, ablation studies, authenticated provenance, human review workflows, and end-to-end testing of downstream tool-execution safety. Containerized deployment and systematic measurement of latency, throughput, scalability, and usability are also needed before operational use.

REFERENCES

- [1] S. Dong, S. Xu, P. He, Y. Li, J. Tang, T. Liu, H. Liu, and Z. Xiang, "Memory injection attacks on LLM agents via query-only interaction," arXiv preprint arXiv:2503.03704, 2025.
- [2] B. D. Sunil, I. Sinha, P. Maheshwari, S. Todmal, S. Mallik, and S. Mishra, "Memory poisoning attack and defence on memory based LLM-agents," arXiv preprint arXiv:2601.05504, 2026.
- [3] S. Patlan, P. Sheng, S. A. Hebbar, P. Mittal, and P. Viswanath, "Real AI agents with fake memories: Fatal context manipulation attacks on Web3 agents," arXiv preprint arXiv:2503.16248, 2025.
- [4] OWASP Foundation, "AI agent security cheat sheet." Accessed: Sep. 25, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/AI_Agent_Security_Cheat_Sheet.html
- [5] S. Abdelnabi and E. Bagdasarian, "AI agents may always fall for prompt injections," arXiv preprint arXiv:2605.17634, 2026.
- [6] OWASP Foundation, "LLM prompt injection prevention cheat sheet." Accessed: Sep. 25, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html
- [7] A. Al Masoud, M. Arazzi, and A. Nocera, "SD-RAG: A prompt-injection-resilient framework for selective disclosure in retrieval-augmented generation," arXiv preprint arXiv:2601.11199, 2026.
- [8] National Institute of Standards and Technology, "Artificial intelligence risk management framework (AI RMF 1.0)," NIST AI 100-1, Jan. 2023. [Online]. Available: <https://doi.org/10.6028/NIST.AI.100-1>
- [9] S. R. Bowman, G. Angeli, C. Potts, and C. D. Manning, "A large annotated corpus for learning natural language inference," in Proc. 2015 Conf. Empirical Methods Natural Language Processing, Lisbon, Portugal, 2015, pp. 632–642.
- [10] National Institute of Standards and Technology, "Secure hash standard (SHS)," FIPS PUB 180-4, Aug. 2015. [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.180-4>
- [11] Ollama, "qwen2.5:3b." Accessed: Sep. 25, 2026. [Online]. Available: <https://ollama.com/library/qwen2.5:3b>
- [12] FastAPI, "FastAPI documentation." Accessed: Sep. 25, 2026. [Online]. Available: <https://fastapi.tiangolo.com/>
- [13] Chroma, "Chroma documentation: Introduction." Accessed: Sep. 25, 2026. [Online]. Available: <https://docs.trychroma.com/docs/overview/introduction>